Hagstofa AI – RAG Backend (Track 1)

This project contains a fully functional **RAG (Retrieval-Augmented Generation)** backend that integrates a local LLM (via Ollama), a vector database (ChromaDB), and a SQLite database for indices and metadata, designed specifically for data from Statistics Iceland (Hagstofa Íslands).

Because the pre-built ChromaDB vector database and SQLite database are large (~190 MB), they are hosted externally via Google Drive to keep the GitHub repository light and within size limits.

🛠️ Requirements & Setup (`requirements.txt`)

Create a file named `requirements.txt` in the root of your repository with the following contents:

```
None
fastapi
uvicorn
chromadb
sentence-transformers
requests
pydantic
pandas
```

System Requirements & Preparation:

1. **Python 3.10+** installed on your machine.
2. **Ollama** installed and running locally with the Llama 3 model:

```
Shell
ollama pull llama3
```

📂 GitHub Repository Structure

Your GitHub repository should look like this:

```
None
hagstofa-ai-track1/
│
├── app.py                        # Main FastAPI application & RAG logic
├── ingest.py                     # Script to fetch and ingest new PxWeb tables
├── requirements.txt              # Python dependencies
├── README.md                     # This documentation file
├── VIS01002_20260810-092754.xlsx # Initial migration dataset reference file
└── .gitignore                    # Files/folders to ignore in Git
```

📥 Step 1: Download the Pre-built Database from Google Drive

To avoid building the vector database from scratch, download the pre-built database archive (`chroma_db_master.zip`):

1. Go to the following link and download the archive: 🔗 **Google Drive - Download chroma_db_master.zip**
2. Extract the contents into the **root** of your project directory (where `app.py` is located). After extraction, you should have:
    - `./chroma_db/` (the vector database directory)
    - `data.db` (the SQLite metadata database)

⚙️ Step 2: Environment Setup

Open a terminal in the project root directory and run the following commands:

1. **Create a virtual environment:**

```
Shell
python -m venv venv
```

2. **Activate the virtual environment:**
    - On Mac / Linux:

```
Shell
source venv/bin/activate
```

    - On Windows (CMD / PowerShell):

```shell
venv\Scripts\activate
```

3. **Install the required packages:**

```shell
pip install --upgrade pip
pip install -r requirements.txt
```

🚀 Step 3: Running the Application

Once your databases and environment are set up, start the FastAPI server:

```shell
uvicorn app:app --reload
```

- The server will run at: `http://127.0.0.1:8000`
- Interactive API Documentation (Swagger UI): `http://127.0.0.1:8000/docs`
- ReDoc Documentation: `http://127.0.0.1:8000/redoc`

🔍 Testing the RAG API via `curl`

You can test the RAG query endpoint using `curl` with a question relevant to the migration dataset included in your pre-built database:

```shell
curl -X POST "http://127.0.0.1:8000/query" \
     -H "Content-Type: application/json" \
     -d '{"prompt": "hvað fluttu margar konur og karlar frá norðurlandi til höfuðborgarinnar árið 2024"}'
```

📈 Step 4: Expanding the System (`ingest.py`)

If you want to fetch *new* tables directly from Statistics Iceland's PxWeb API and add them to your ChromaDB database later, use the `ingest.py` script.

Create a file named `ingest.py` with the following code:

Python

```python
import os
import requests
import json
import chromadb
from sentence_transformers import SentenceTransformer

# ------------------------------------------------------------------------
# CONFIGURATION: Insert the PxWeb API URL and JSON query manually here
# ------------------------------------------------------------------------
PXWEB_API_URL =
"https://px.hagstofa.is/pxis/api/v1/is/Ibuar/mannfjoldi/2_byggdir/sveitarfelog/MAN
02001.px"

PXWEB_QUERY = {
  "query": [
    {
      "code": "Sveitarfélög",
      "selection": {
        "filter": "all",
        "values": ["*"]
      }
    }
  ],
  "response": {
    "format": "json-stat2"
  }
}

def fetch_data_from_pxweb(url: str, query: dict) -> list[dict]:
    print(f"Fetching data from PxWeb API: {url}...")
    response = requests.post(url, json=query)

    if response.status_code != 200:
        raise Exception(f"Failed to fetch data: {response.status_code} -
{response.text}")

    data = response.json()
    dataset_title = data.get("label", "Statistics Iceland Dataset")

    text_content = f"Table Title: {dataset_title}\nInformation from PxWeb API
query."

    documents = [{
        "id": url.split("/")[-1].replace(".px", ""),
        "text": text_content,
        "raw_json": json.dumps(data, ensure_ascii=False)
    }]
```

```
    return documents

def run_ingestion():
    print("1. Loading embedding model (all-MiniLM-L6-v2)...")
    model = SentenceTransformer('all-MiniLM-L6-v2')

    if not os.path.exists("./chroma_db"):
        print("⚠️ Warning: './chroma_db' folder not found. Please download the
database from Google Drive first!")
        return

    client = chromadb.PersistentClient(path="./chroma_db")
    collection = client.get_or_create_collection(name="hagstofa_rag")

    try:
        new_entries = fetch_data_from_pxweb(PXWEB_API_URL, PXWEB_QUERY)
    except Exception as e:
        print(f"❌ Ingestion failed: {e}")
        return

    texts = [item["text"] for item in new_entries]
    ids = [item["id"] for item in new_entries]
    metadatas = [{"raw_json": item["raw_json"]} for item in new_entries]

    print("2. Generating embeddings for new data...")
    embeddings = model.encode(texts).tolist()

    print("3. Upserting data into ChromaDB...")
    collection.upsert(
        documents=texts,
        embeddings=embeddings,
        metadatas=metadatas,
        ids=ids
    )

    print("✅ Ingestion and embedding successful! New tables have been added to
the vector database.")

if __name__ == "__main__":
    run_ingestion()
```

To execute the ingestion script and add new datasets:

```
Shell
python ingest.py
```

🛑 .gitignore

To ensure large local binary files or virtual environments are not tracked by Git, include a `.gitignore` file with:

```
None
venv/
__pycache__/
*.pyc
.env
```